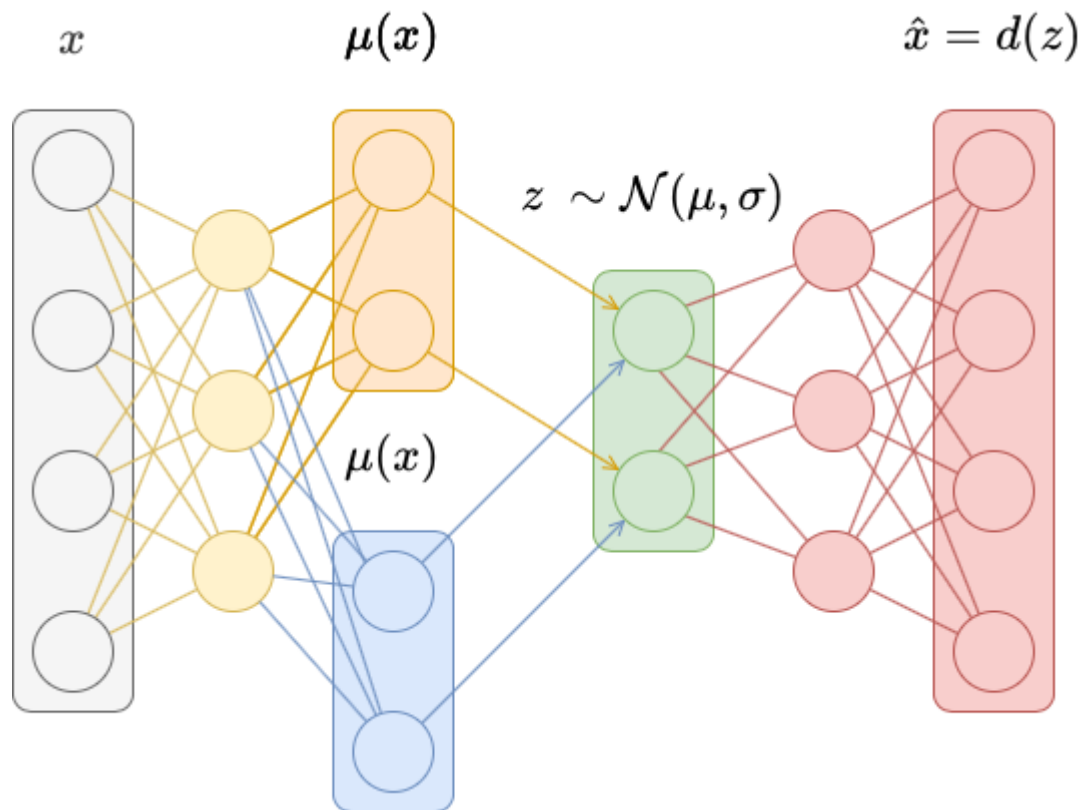


Demystifying Neural Networks: Variational AutoEncoders



Introduction

In the realm of deep learning, autoencoders are well known for their ability to compress and reconstruct data. Their goal is to learn efficient representations of input data, usually serving purposes like dimensionality reduction or feature extraction. Enter the Variational Autoencoder (VAE), a close relative of the traditional autoencoder, but one infused with a touch of probabilistic magic. This special twist makes VAEs incredibly powerful generative models.

What is a Variational Autoencoder (VAE)?

A variational autoencoder is a type of generative neural network architecture. At its heart, a VAE still has the same structural components as a traditional autoencoder: an encoder and a decoder.

- **Encoder:** The encoder's job is to take an input (like an image) and compress it into a compact, latent representation. Traditional autoencoders output a single point in the latent space. A VAE, however, outputs a *distribution* (usually a Gaussian) over the latent space.
- **Decoder:** The decoder samples a point from this distribution in the latent space and attempts to reconstruct the original input as accurately as possible.

Why Use a VAE?

- **Generative Power:** The key advantage of VAEs lies in their generative capabilities. Because a VAE learns to model the underlying probability distribution of the input data, it can **generate new samples** that are similar to the original data. Think generating realistic faces, composing new pieces of music, or crafting creative text.
- **Continuous Latent Space:** Unlike traditional autoencoders, VAEs promote a continuous and smooth latent space. This means that we can interpolate between points in this space to generate smoothly transitioning variations of our data. Imagine smoothly morphing one person's face into another's!

VAE vs. Autoencoder: Differences

Variational Autoencoder (VAE)

- **Latent Space:** Continuous and smooth, enabling interpolation and meaningful transitions of representations.

- **Generative Power:** Powerful generative capabilities to produce new, similar data samples.
- **Loss:** Reconstruction loss combined with KL divergence to regularize the latent space, forcing it to follow a prior distribution (often Gaussian).

Autoencoder (AE)

- **Latent Space:** Potentially discrete, which can lead to gaps or abrupt changes in representations.
- **Generative Power:** More limited in generating entirely new data samples.
- **Loss:** Primarily focused on reconstruction loss, ensuring accurate reproduction of the input.

How Does a VAE Work?

1. **The Distribution Trick:** Instead of a point in the latent space, the encoder of a VAE outputs the parameters that define a probability distribution (usually mean and variance). During training, we sample a point from this distribution to feed into the decoder.
2. **The Reparameterization Trick:** This is the clever part. Directly backpropagating gradients through random sampling is tricky. The reparameterization trick lets us express the sampled point in the latent space as a deterministic function of the distribution parameters and an external random variable. This allows for proper training.
3. **Loss Function: Beyond Reconstruction:** The VAE's loss function has two parts:

- **Reconstruction Loss:** Just like in an autoencoder, this part ensures that the decoder accurately reconstructs the input.
- **KL Divergence:** This is where the probabilistic twist comes in. The Kullback-Leibler (KL) divergence measures how much the encoder's learned distribution diverges from a standard prior distribution (often a standard Gaussian). This encourages a well-structured, regular latent space.

The Reparameterization Trick

The core difficulty lies in the concept of backpropagation, a training technique in neural networks. Backpropagation allows us to adjust the network's internal parameters based on the difference between the predicted output and the actual output. However, it struggles when dealing with random sampling, which is inherent in VAEs.

The Traditional (Broken) Approach

Imagine the encoder outputs the mean (μ) and standard deviation (σ) of a Gaussian distribution. We then sample a random variable from the distribution as:

$z = \text{random_sample}(\mu, \sigma)$

This seems straightforward, but there's a catch. Backpropagation requires calculating the gradients with respect to the model's parameters (μ and σ in this case). However, calculating the gradient with respect to z (a randomly sampled variable) is mathematically undefined. This throws a wrench in the training process.

The reparameterization trick offers an elegant solution. Instead of directly sampling z from its distribution $N(\mu, \sigma^2)$, we **decompose z into a deterministic component and a stochastic component** that is independent of the parameters we want to optimize.

Here's the magic:

1. **Introduce a new random variable:** We introduce another independent random variable, typically another standard normal variable (ϵ). This new variable (ϵ) serves as a source of randomness entirely separate from the sampling process within the VAE.
2. **Reparametrize the latent variable:** Instead of directly adding noise, we create a scaled version of the new random variable (ϵ) using the standard deviation (σ) predicted by the encoder. We achieve this with element-wise multiplication:

$$z = \mu + \sigma * \epsilon$$

This equation might seem very similar to the original equation, but there's a crucial difference. Now, both μ and σ are deterministic outputs from the encoder, and ϵ is a separate source of randomness we can control.

Why Does This Work?

Even though new equation introduces a new variable (ϵ), it still captures the essence of the original Gaussian distribution. The scaled ϵ effectively injects randomness while maintaining the relationship between the mean (μ) and standard deviation (σ) defined by the encoder.

Benefits:

- **Backpropagation is Happy:** Because the new equation only involves deterministic operations, we can now calculate gradients with respect to μ and σ during backpropagation. This allows the network to learn and adjust its parameters effectively.

- **Preserves Stochasticity:** Even though the reparameterization trick uses a deterministic transformation, the final sampled latent variable (z) retains the desired statistical properties of the original Gaussian distribution. This ensures the VAE maintains its ability to explore the latent space effectively.

The reparameterization trick is a brilliant illustration of how a seemingly simple mathematical transformation can have a profound impact on the training process of complex neural network architectures like VAEs.

Introduction

Variational Autoencoders (VAEs) are generative models explicitly designed to capture the underlying probability distribution of a given dataset and generate novel samples. They utilize an architecture that comprises an encoder-decoder structure. The encoder transforms input data into a latent form, and the decoder aims to reconstruct the original data based on this latent representation. The VAE is programmed to minimize the dissimilarity between the original and reconstructed data, enabling it to comprehend the underlying data distribution and generate new samples that conform to the same distribution.

One notable advantage of VAEs is their ability to generate new data samples resembling the training data. Because the VAE's latent space is continuous, the decoder can generate new data points that seamlessly interpolate among the training data points. VAEs find applications in various domains like density estimation and text generation.

The Architecture of Variational Autoencoder

A VAE typically has two major components: An [encoder](#) connection and a decoder connection. An encoder network transforms the input data into a low-dimensional secret space, often called a “secret code”.

Various [neural network](#) topologies, such as fully connected or convolutional neural networks, can be investigated for implementing encoder networks. The architecture chosen is based on the characteristics of the data. The encoder network produces essential parameters, such as the mean and variance of a Gaussian distribution, necessary for sampling and generating the latent code.

Similarly, researchers can construct the decoder network using various types of neural networks, and its objective is to reconstruct the original data from the provided latent code.

Example of VAE architecture:

VAE comprises an encoder network that maps input data to a latent code and a decoder network that conducts the inverse operation by translating the latent code back to the reconstruction data. By undergoing this training process, the VAE learns an optimized latent representation that captures the fundamental characteristics of the data, enabling precise reconstruction.

Intuitions About the Regularization

In addition to the architectural aspects, researchers apply regularization to the latent code, making it a vital element of VAEs. This regularization prevents overfitting by encouraging a smooth distribution of the latent code rather than simply memorizing the training data.

The regularization not only aids in generating new data samples that interpolate smoothly between training data points but also contributes to the VAE’s ability to generate novel data resembling the training data. Moreover, this regularization prevents the decoder network from perfectly reconstructing the input data, promoting the learning of a more

general data representation that enhances the VAE's capacity for generating diverse data samples.

Mathematically, in VAEs, researchers express the regularization by incorporating a Kullback-Leibler (KL) divergence term into the loss function. The encoder network generates parameters (e.g., mean and log-variance) of a Gaussian distribution for sampling the latent code. The loss function of a VAE includes the calculation of the KL divergence between the distribution of the learned latent variables and a prior distribution, normal distribution. Researchers incorporate the KL divergence term to encourage the latent variables to possess distributions similar to the prior distribution.

here is the formula for KL divergence:

$$KL(q(z|x)||p(z)) = E[\log q(z|x) - \log p(z)]$$

In summary, the regularization incorporated in VAEs plays a crucial role in enhancing the model's capacity to generate fresh data samples while mitigating the risk of overfitting the training data.

Mathematical Details of VAEs

Probabilistic Framework and Assumptions

The probabilistic framework of a VAE can be outlined as follows:

Latent Variables

The inclusion of keywords such as “latent distribution,” “latent variable z ,” “deep generative models,” and “random variable” is pivotal in facilitating their incorporation within a model structured around a simpler (usually exponential) conditional distribution pertaining to the observable variable. This setup revolves around a probability distribution

involving two variables: $p(x, z)$. While the variable x is readily observable in the dataset being analyzed, the variable z remains concealed. The overall probability distribution can be expressed as $p(x, z) = p(x|z)p(z)$.

Observed Variables

We have an observed variable x , which is assumed to follow a likelihood distribution $p(x|z)$ (for example, a Bernoulli distribution).

Likelihood Distribution

$L(x, z)$ is a function that depends on two variables. If we set the value of x , the likelihood function can be understood as a distribution representing the probability distribution of z for that particular fixed x . However, if we set the value of z , the likelihood function should not be regarded as a distribution for x . In most cases, it does not adhere to the characteristics of a distribution, such as summing up to 1. Nevertheless, certain scenarios exist where the likelihood function can formally meet the distribution criteria and satisfy the requirement of summing to 1.

The combined distribution of the latent and observable variables is as follows: $p(x, z) = p(x|z)p(z)$. A joint probability distribution presents the probability distribution for multiple random variables.

The main purpose of a VAE is to understand the true posterior distribution of the latent variables, denoted as $p(z|x)$. A VAE accomplishes this by employing an encoder network to approximate the genuine posterior distribution with a learned approximation $q(z|x)$.

Posterior Distribution

In Bayesian statistics, a posterior probability refers to the adjusted or updated probability of an event happening in light of newly acquired information. Update the prior probability by applying Bayes' theorem to calculate the posterior probability.

The VAE learns the model parameters by maximizing the Evidence Lower Bound (ELBO):

$$\text{ELBO} = \mathbb{E}[\log(p(x|z))] - \text{KL}(q(z|x)||p(z))$$

ELBO consists of two terms. The first term is the reconstruction term, which calculates the ability of the VAE to recover the input data correctly. The second term, the KL variance, defines the difference between the estimated posterior distribution ($q(z|x)$) and the prior distribution ($p(z)$).

By employing a probabilistic framework, VAE models generate the data assuming that the input data from a latent space is on specific probabilistic distributions. The objective is to learn the true posterior distribution by maximizing the likelihood of the input data.

Variational Inference Formulation

The formulation of Variational Inference in a VAE is as follows:

- **Approximate posterior distribution:** We have an approximation of the posterior distribution $q(z|x)$.
- **True posterior distribution:** We have the true posterior distribution $p(z|x)$.

The aim is to find a similar distribution ($q(z|x)$) that approximates the true distribution ($p(z|x)$) as closely as possible, using the KL divergence method.

The KL variance equation compares two probability distributions, $q(z|x)$ and $p(z|x)$, to measure their differences.

During VAE training, we try to minimize the KL divergence by increasing the evidence of lower boundary (ELBO), a combination of the reconstruction term and the KL divergence. The reconstruction term assesses the model's ability to reconstruct input data, while the KL divergence measures the difference between the approximate and actual distributions.

Assumptions:

1. **Latent Variable Assumption:** We assume that each data point x is generated by a latent variable z sampled from a latent space Z .
2. **Latent Space Prior:** We assume a prior distribution over the latent space, typically a multivariate Gaussian distribution.

Model:

1. **Encoder:** The encoder maps the input data x to a distribution over the latent space Z . It outputs the parameters of this distribution, typically the mean μ and variance σ^2 of a Gaussian distribution.

$$q_{\phi}(z|x) = \mathcal{N}(\mu_{\phi}(x), \sigma_{\phi}^2(x))$$

where ϕ denotes the parameters of the encoder.

2. **Decoder:** The decoder takes a sample z from the distribution over the latent space and maps it back to the input space.

$$p_{\theta}(x|z)$$

where θ denotes the parameters of the decoder.

3. **Variational Inference:** VAEs employ variational inference to approximate the true posterior $p(z|x)$ with a simpler distribution $q_{\phi}(z|x)$. This is done by minimizing the Kullback-Leibler (KL) divergence between the approximate posterior and the prior:

$$\text{KL}(q_{\phi}(z|x)||p(z|x)) = \frac{1}{2} \sum_{j=1}^J (1 + \log((\sigma_j)^2) - (\mu_j)^2 - (\sigma_j)^2)$$

where J is the dimensionality of the latent space.

Objective:

The VAE objective consists of two terms: reconstruction loss and KL divergence:

$$\mathcal{L}(\theta, \phi; x) = \mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x|z)] - \text{KL}(q_\phi(z|x) || p(z))$$

1. **Reconstruction Loss:** Measures how well the model reconstructs the input data. It is typically chosen to be the negative log-likelihood of the data given the model output, often binary cross-entropy for image data.

$$\mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x|z)]$$

2. **KL Divergence:** Measures the divergence between the approximate posterior and the prior over the latent space. It encourages the approximate posterior to be close to the prior.

$$\text{KL}(q_\phi(z|x) || p(z))$$

Training:

During training, we optimize the parameters θ and ϕ of the decoder and encoder, respectively, to maximize the variational lower bound $\mathcal{L}(\theta, \phi; x)$.

Sampling:

After training, we can sample from the learned latent space to generate new data points by sampling from the prior distribution over the latent space $p(z)$ and passing the samples through the decoder.